

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4&5

The Core of OOP & Encapsulation



Disusun Oleh:

Samuel Reinaldo 105224013

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Sistem reservasi pada jaringan perhotelan saat ini rentan terhadap kendala operasional, seperti kelolosan jumlah tamu yang melebihi kapasitas (overcapacity) hingga penyalahgunaan kode voucher diskon. Jika dibiarkan, celah ini dapat mengganggu operasional dan merugikan manajemen secara finansial.

Proyek ini bertujuan merancang ulang arsitektur sistem reservasi (blueprint kelas KamarHotel) agar bertindak sebagai "pengawas" otomatis yang kebal terhadap kesalahan input (human error). Pendekatan utama yang digunakan berfokus pada dua pilar Object-Oriented Programming (OOP): Enkapsulasi dan Overloading Metode.

Melalui pembaruan ini, diharapkan operasional meja resepsionis perhotelan dapat berjalan jauh lebih presisi, aman, dan efisien.

II. Variabel

No	Nama Variabel	Tipe Data
1	nomorKamar	String
2	tipeKamar	String
3.	kapasitasMaksimal	Int
4.	hargaPerMalam	double
5.	isTersedia	Boolean
6.	jumlahTamu	Int
7.	jumlahMalam	Int
8.	kodeVoucher	String

9.	totalNormal	Double
10.	diskon	double
11.	kamar1, kamar2	boolean
12.	tagihanKamar1, tagihanKamar2	double

iii. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal)	constructor	Membuat objek kamar baru dengan pendaftaran kilat. Harga otomatis diset 0 dan status kamar diset true (tersedia).
2	KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam)	constructor	Membuat objek kamar baru dengan pendaftaran lengkap beserta input harga per malam. Status kamar diset <i>true</i> .
3.	getNomorKamar(), getTipeKamar(), getKapasitasMaksimal(), getHargaPerMalam(), getIsTersedia()	Functional	Fungsinya sebagai getter untuk membaca/mengambil nilai dari masing masing atribut yang sudah dienkapsulasi.
4.	setTipeKamar(String)	procedural	Mengubah nilai tipe kamar dengan validasi bisnis yang ketat (hanya menerima input "Reguler", "Premium", atau "Suite").
5.	setHargaPerMalam(double)	procedural	Mengubah tarif harga per malam dengan validasi bisnis batas bawah (menolak angka di bawah 50.000).
6.	setNomorKamar(String), setKapasitasMaksimal(int)	procedural	Berfungsi sebagai pintu akses (Setter) untuk memodifikasi atau mengubah nilai atribut secara standar.
7.	batalPesan()	procedural	Membatalkan reservasi dengan mengubah status atribut isTersedia kembali menjadi true.
8.	pesanKamar()	Procedural	Melakukan pemesanan standar dengan mengubah status kamar menjadi tidak tersedia (false)
9.	pesanKamar(int jumlahTamu)	Procedural	Melakukan pemesanan dengan validasi ekstra untuk mengecek apakah jumlah tamu diinput melebihi kapasitas maksimal kamar (<i>overcapacity</i>).

10.	hitungTotalBayar(int jumlahMalam)	functional	Mengkalkulasi dan mengembalikan nilai total tagihan menginap dengan harga standar/normal.
11.	hitungTotalBayar(int jumlahMalam, string kodeVoucher)	Functional	Mengkalkulasi tagihan dengan validasi promo. Mengembalikan nilai yang sudah didiskon 20% jika syarat terpenuhi.
12.	tampilkanProfilKamar()	procedural	Menampilkan susunan cetak struk/profil yang berisi seluruh informasi detail atribut kamar ke layar komputer.

IV. Dokumentasi dan Pembahasan Code

Di bagian KamarHotel.java:

```
1  class KamarHotel {
2
3      // Encapsulation
4      private String nomorKamar;
5      private String tipeKamar;
6      private int kapasitasMaksimal;
7      private double hargaPerMalam;
8      private boolean isTersedia;
9
10     public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
11         this.nomorKamar = nomorKamar;
12         setTipeKamar(tipeKamar);
13         this.kapasitasMaksimal = kapasitasMaksimal;
14         this.hargaPerMalam = 0.0;
15         this.isTersedia = true;
16     }
17
18     public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
19         this.nomorKamar = nomorKamar;
20         setTipeKamar(tipeKamar);
21         this.kapasitasMaksimal = kapasitasMaksimal;
22         setHargaPerMalam(hargaPerMalam);
23         this.isTersedia = true;
24     }
25
26     public String getNomorKamar() { return nomorKamar; }
27     public void setNomorKamar(String nomorKamar) { this.nomorKamar = nomorKamar; }
28
29     public String getTipeKamar() { return tipeKamar; }
30
31     public void setTipeKamar(String tipeKamar) {
32         if (tipeKamar.equalsIgnoreCase("Reguler") ||
33             tipeKamar.equalsIgnoreCase("Premium") ||
34             tipeKamar.equalsIgnoreCase("Suite")) {
35
36             this.tipeKamar = tipeKamar.substring(0, 1).toUpperCase() + tipeKamar.substring(1).toLowerCase();
37         } else {
38             System.out.println("[SISTEM TOLAK] Tipe '" + tipeKamar + "' tidak dikenali. Dipaksa menjadi 'Reguler'.");
39             this.tipeKamar = "Reguler";
40         }
41     }
42
43     public int getKapasitasMaksimal() { return kapasitasMaksimal; }
44     public void setKapasitasMaksimal(int kapasitasMaksimal) { this.kapasitasMaksimal = kapasitasMaksimal; }
45
46     public double getHargaPerMalam() { return hargaPerMalam; }
47 }
```

```

48 // Menetapkan harga tiap malam
49 public void setHargaPerMalam(double hargaPerMalam) {
50     if (hargaPerMalam >= 50000) {
51         this.hargaPerMalam = hargaPerMalam;
52     } else {
53         System.out.println("[SISTEM TOLAK] Harga tidak boleh di bawah 50000. Dipaksa menjadi 50000.");
54         this.hargaPerMalam = 50000;
55     }
56 }
57
58 public boolean getIsTersedia() { return isTersedia; }
59
60 public void batalPesan() {
61     this.isTersedia = true;
62     System.out.println("[INFO] Pesanan kamar " + nomorKamar + " dibatalkan. Kamar kini tersedia.");
63 }
64
65 // Reservasi
66 public void pesanKamar() {
67     if (isTersedia) {
68         this.isTersedia = false;
69         System.out.println("[SUKSES] Kamar " + nomorKamar + " berhasil dipesan.");
70     } else {
71         System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi!");
72     }
73 }
74
75 public void pesanKamar(int jumlahTamu) {
76     if (!isTersedia) {
77         System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi!");
78         return;
79     }
80     if (jumlahTamu <= kapasitasMaksimal) {
81         this.isTersedia = false;
82         System.out.println("[SUKSES] Kamar " + nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " tamu.");
83     } else {
84         System.out.println("[TEGURAN] Overcapacity! Kamar " + nomorKamar + " hanya muat " + kapasitasMaksimal + " orang. Pesanan ditolak.");
85     }
86 }

```

```

88 // Bill
89
90 // Menghitung total tanpa voucher
91 public double hitungTotalBayar(int jumlahMalam) {
92     return hargaPerMalam * jumlahMalam;
93 }
94
95 // Menghitung total bayar dengan voucher (promo 20% (minimal 3 malam)
96 public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
97     double totalNormal = hitungTotalBayar(jumlahMalam);
98     if (kodeVoucher.equals("PROMO") && jumlahMalam >= 3) {
99         double diskon = totalNormal * 0.20;
100         return totalNormal - diskon;
101     } else {
102         System.out.println("[INFO VOUCHER] Gagal pakai voucher: " + kodeVoucher + ". Syarat min 3 malam tidak terpenuhi atau kode salah. Tagihan normal berlaku.");
103         return totalNormal;
104     }
105 }
106
107 // Cetak profil kamar
108 public void tampilkanProfilKamar() {
109     System.out.println("-----");
110     System.out.println("Nomor Kamar : " + nomorKamar);
111     System.out.println("Tipe : " + tipeKamar);
112     System.out.println("Kapasitas : " + kapasitasMaksimal + " Orang");
113     System.out.println("Harga/Malam : Rp " + hargaPerMalam);
114     System.out.println("Status : " + (isTersedia ? "Tersedia" : "Terisi"));
115     System.out.println("-----");
116 }
117 }

```

Di bagian Main.java:

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println("=== MEMULAI SIMULASI SISTEM HOTEL ===\n");
4
5         KamarHotel kamar1 = new KamarHotel("101", "Reguler", 2);
6
7         KamarHotel kamar2 = new KamarHotel("202A", "Suite", 2, 750000);
8
9         System.out.println(">>> UJI COBA INPUT DATA NGAWUR (Kamar 1)");
10        kamar1.setTipeKamar("Presidential");
11        kamar1.setHargaPerMalam(-10000);
12        System.out.println();
13
14        System.out.println(">>> UJI COBA OVERCAPACITY (Kamar 2)");
15        kamar2.pesanKamar(4);
16        System.out.println();
17
18        System.out.println(">>> UJI COBA PESAN SESUAI ATURAN (Kamar 2)");
19        kamar2.pesanKamar(2);
20        System.out.println();
21
22        System.out.println(">>> UJI COBA DOUBLE BOOKING (Kamar 2)");
23        kamar2.pesanKamar();
24        System.out.println();
25
26        System.out.println(" UJI COBA TAGIHAN & VOUCHER");
27        double tagihanKamar1 = kamar1.hitungTotalBayar(2, "PROMO");
28        System.out.println("Total Tagihan Kamar 1 (2 Malam): Rp" + tagihanKamar1);
29
30        System.out.println();
31
32        double tagihanKamar2 = kamar2.hitungTotalBayar(4, "PROMO");
33        System.out.println("Total Tagihan Kamar 2 (4 Malam, Diskon 20%): Rp" + tagihanKamar2);
34        System.out.println();
35
36        System.out.println("=== STRUK INFORMASI PROFIL KAMAR (KONDISI AKHIR) ===");
37        kamar1.tampilkanProfilKamar();
38        kamar2.tampilkanProfilKamar();
39    }
40 }

```

Pembahasan Kode

```
1  class KamarHotel {  
2  
3      // Encapsulation  
4      private String nomorKamar;  
5      private String tipeKamar;  
6      private int kapasitasMaksimal;  
7      private double hargaPerMalam;  
8      private boolean isTersedia;
```

Kode dimulai dengan membuat blueprint (class) bernama KamarHotel. Kemudian, di dalamnya kita mendeklarasikan lima buah atribut yang dikunci secara private (penerapan Encapsulation). Atribut-atribut tersebut adalah nomorKamar dan tipeKamar dengan tipe data String, kapasitasMaksimal dengan tipe data int, hargaPerMalam dengan tipe data double, serta isTersedia dengan tipe data boolean.

```
10     public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {  
11         this.nomorKamar = nomorKamar;  
12         setTipeKamar(tipeKamar);  
13         this.kapasitasMaksimal = kapasitasMaksimal;  
14         this.hargaPerMalam = 0.0;  
15         this.isTersedia = true;  
16     }  
17  
18     public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {  
19         this.nomorKamar = nomorKamar;  
20         setTipeKamar(tipeKamar);  
21         this.kapasitasMaksimal = kapasitasMaksimal;  
22         setHargaPerMalam(hargaPerMalam);  
23         this.isTersedia = true;  
24     }
```

Setelah kita mendeklarasikan apa saja yang kita butuhkan dalam KamarHotel ini, kita membuat constructor. Ada 2 constructor yaitu constructor atas yang hanya menerima 3 parameter (nomorKamar, tipeKamar, kapasitasMaksimal) dan constructor bawah yang menerima 4 parameter (nomorKamar, tipeKamar, kapasitasMaksimal, hargaPerMalam).

Fungsi constructor di atas Adalah untuk pendaftaran kilat. Cara kerjanya Adalah sistem menginisiasi data tersebut dan secara otomatis menetapkan hargaPerMalam menjadi 0.0 dan status isTersedia menjadi True. Sementara, constructor bawah dirancang untuk menerima data yang lebih lengkap dengan menambahkan hargaPerMalam ke dalamnya. Hal itu dilakukan agar hargaPerMalam bisa langsung diisi oleh pengguna sejak awal.

Hal menarik dalam kedua proses inisiasi di atas Adalah kita menggunakan kata kunci this untuk memasukkan nilai secara langsung, kita juga memanggil fungsi Setter (seperti setTipeKamar dan setHargaPerMalam) agar

data yang masuk bisa melewati proses pengecekan atau validasi terlebih dahulu sebelum benar-benar disimpan.

```
26 public String getNomorKamar() { return nomorKamar; }
27 public void setNomorKamar(String nomorKamar) { this.nomorKamar = nomorKamar; }
28
29 public String getTipeKamar() { return tipeKamar; }
30
31 public void setTipeKamar(String tipeKamar) {
32     if (tipeKamar.equalsIgnoreCase("Reguler") ||
33         tipeKamar.equalsIgnoreCase("Premium") ||
34         tipeKamar.equalsIgnoreCase("Suite")) {
35
36         this.tipeKamar = tipeKamar.substring(0, 1).toUpperCase() + tipeKamar.substring(1).toLowerCase();
37     } else {
38         System.out.println("[SISTEM TOLAK] Tipe '" + tipeKamar + "' tidak dikenali. Dipaksa menjadi 'Reguler'.");
39         this.tipeKamar = "Reguler";
40     }
41 }
```

Dalam bagian ini, kita menginisiasi variable nya satu per satu. Mulai dari kita mengatur variable nomorKamar. Kita menggunakan getter dalam nomorKamar untuk membaca/mengambil nilai yang sudah dienkapsulasi, lalu kita menggunakan setter untuk memodifikasi nomor kamar secara standar. Hal kedua yang kita lakukan adalah kita mengatur variable tipeKamar. Kita menggunakan getter dalam tipeKamar untuk membaca/mengambil nilai yang sudah dienkapsulasi, lalu kita menggunakan setter untuk mengubah kategori tipe kamar sesuai dengan validasi bisnis. Dalam bagian setTipeKamar, sistem dirancang untuk hanya menerima input kategori yang sah (Reguler, Premium, atau Suite) dan secara otomatis menolak serta memperbaiki input yang salah.

```
43 public int getKapasitasMaksimal() { return kapasitasMaksimal; }
44 public void setKapasitasMaksimal(int kapasitasMaksimal) { this.kapasitasMaksimal = kapasitasMaksimal; }
45
46 public double getHargaPerMalam() { return hargaPerMalam; }
47
48 // Menetapkan harga tiap malam
49 public void setHargaPerMalam(double hargaPerMalam) {
50     if (hargaPerMalam >= 50000) {
51         this.hargaPerMalam = hargaPerMalam;
52     } else {
53         System.out.println("[SISTEM TOLAK] Harga tidak boleh di bawah 50000. Dipaksa menjadi 50000.");
54         this.hargaPerMalam = 50000;
55     }
56 }
57
58 public boolean getIsTersedia() { return isTersedia; }
59
60 public void batalPesan() {
61     this.isTersedia = true;
62     System.out.println("[INFO] Pesanan kamar " + nomorKamar + " dibatalkan. Kamar kini tersedia.");
63 }
```

Hal ketiga yang kita lakukan Adalah mengatur variable kapasitasMaksimal. Kita menggunakan getter dalam kapasitasMaksimal untuk membaca/mengambil nilai yang sudah dienkapsulasi, lalu kita menggunakan setter untuk memodifikasi kapasitas maksimal secara standar.

Hal keempat yang kita lakukan Adalah kita mengatur variable hargaPerMalam. Kita menggunakan getter dalam hargaPerMalam untuk membaca/mengambil nilai yang sudah dienkapsulasi, lalu kita menggunakan setter untuk menetapkan harga per malam sesuai dengan validasi bisnis. Dalam sethargaPerMalam ini, modifikasi data tidak dibiarkan begitu saja, harus melewati validasi bisnis yang ketat. Sistem dirancang untuk hanya menerima harga tepat 50rb atau 50.001 seterusnya, tidak dapat menerima harga di bawah 50rb. Jika menetapkan harga di bawah 50rb, maka sistem akan menolak dan memaksa menjadi 50rb.

Hal kelima yang kita lakukan Adalah kita mengatur variable isTersedia. Kita menggunakan getter dalam kapasitasMaksimal untuk membaca dan mengembalikan status kamar saat ini (apakah bernilai true yang berarti tersedia, atau false yang berarti sedang terisi).lalu kita menggunakan setter untuk memodifikasi kapasitas maksimal secara standar.

Untuk bagian batalpesan(), cara kerjanya Adalah ketika dieksekusi, metode ini akan membatalkan reservasi tamu dengan mengubah status isTersedia kembali menjadi true, lalu mencetak pesan informasi ke layar komputer bahwa kamar tersebut kini sudah kosong dan siap dipesan kembali.

```
65 // Reservasi
66 public void pesanKamar() {
67     if (isTersedia) {
68         this.isTersedia = false;
69         System.out.println("[SUKSES] Kamar " + nomorKamar + " berhasil dipesan.");
70     } else {
71         System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi!");
72     }
73 }
74
75 public void pesanKamar(int jumlahTamu) {
76     if (!isTersedia) {
77         System.out.println("[GAGAL] Kamar " + nomorKamar + " sudah terisi!");
78         return;
79     }
80     if (jumlahTamu <= kapasitasMaksimal) {
81         this.isTersedia = false;
82         System.out.println("[SUKSES] Kamar " + nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " tamu.");
83     } else {
84         System.out.println("[TEGURAN] Overcapacity! Kamar " + nomorKamar + " hanya muat " + kapasitasMaksimal + " orang. Pesanan");
85     }
86 }
```

Dalam kode di atas, terdapat 2 cara pemesanan kamar. Cara pertama adalah pemesanan kilat tanpa syarat khusus (pesanKamar()); sistem hanya mengecek apakah kamar masih kosong, dan jika ya, status kamar langsung diubah menjadi terisi (false). Cara kedua adalah pemesanan yang lebih ketat (pesanKamar(jumlahTamu)); di mana sistem tidak hanya memastikan kamarnya kosong, tetapi juga memeriksa apakah jumlah tamu yang mau menginap tidak melebihi batas maksimal kapasitas kamar. Jika kamarnya muat, pesanan sukses dicatat, namun jika tamunya terlalu banyak, sistem akan otomatis memblokir pesanan tersebut dan mengeluarkan teguran overcapacity.

```

88 // Bill
89
90 // Menghitung total tanpa voucher
91 public double hitungTotalBayar(int jumlahMalam) {
92     return hargaPerMalam * jumlahMalam;
93 }
94
95 // Menghitung total bayar dengan voucher promo 20% (minimal 3 malam)
96 public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
97     double totalNormal = hitungTotalBayar(jumlahMalam);
98     if (kodeVoucher.equals("PROMO") && jumlahMalam >= 3) {
99         double diskon = totalNormal * 0.20;
100         return totalNormal - diskon;
101     } else {
102         System.out.println("[INFO VOUCHER] Gagal pakai voucher '" + kodeVoucher + "'. Syarat min 3 malam tidak terpenuhi atau k");
103         return totalNormal;
104     }
105 }

```

Di bagian pembayaran ini, ada 2 macam pembayaran, yang menggunakan voucher dan yang tidak menggunakan voucher. Yang tidak menggunakan voucher, perhitungan bayarnya hanya hargaPerMalam dikali dengan jumlahMalam. Sementara untuk yang menggunakan voucher, perhitungannya Adalah pertama menyimpan hitungtotalBayar menjadi totalNormal (totalNormal yang akan digunakan untuk perhitungan pada saat sudah bersama diskon). Kedua voucher akan menyeleksi terlebih dahulu apakah kedua syarat dipenuhi, jika kedua syarat tidak dipenuhi, maka sistem akan memberitahu bahwa gagal menggunakan voucher. Sementara jika syarat dipenuhi, sistem akan lanjut menghitung diskon dengan rumus totalNormal dikali 0.20. Lalu perhitungan terakhir yaitu totalNormal dikurangi dengan diskon.

```

107 // Cetak profil kamar
108 public void tampilkanProfilKamar() {
109     System.out.println("-----");
110     System.out.println("Nomor Kamar : " + nomorKamar);
111     System.out.println("Tipe : " + tipeKamar);
112     System.out.println("Kapasitas : " + kapasitasMaksimal + " Orang");
113     System.out.println("Harga/Malam : Rp" + hargaPerMalam);
114     System.out.println("Status : " + (isTersedia ? "Tersedia" : "Terisi"));
115     System.out.println("-----");
116 }
117 }

```

Kita membuat sebuah metode bernama tampilkanProfilKamar() yang bertugas seperti "mesin cetak struk". Metode ini murni melakukan aksi tanpa mengembalikan nilai perhitungan apa pun ke sistem (ditandai dengan kata void). Cara kerjanya sangat sederhana: ia menggunakan perintah System.out.println untuk menyusun dan menampilkan seluruh data kamar, mulai dari nomor, tipe, kapasitas, hingga harga ke layar komputer agar rapi dan mudah dibaca oleh staf hotel.

Ini Adalah bagian Main.java:

```

3     System.out.println("== MEMULAI SIMULASI SISTEM HOTEL ==\n");
4
5     KamarHotel kamar1 = new KamarHotel("101", "Reguler", 2);
6
7     KamarHotel kamar2 = new KamarHotel("202A", "Suite", 2, 750000);

```

Hal pertama yang kita lakukan dalam Main.java Adalah kita mencetak "=== MEMULAI SIMULASI SISTEM HOTEL ===" menggunakan perintah `system.out.println`. Lalu kita langsung mengimplementasikan hasil dari kode yang kita buat di KamarHotel.java. Kita membuat 2 kamar. Kamar 1 dibuat menggunakan "pendaftaran kilat" yang hanya mengisi nomor, tipe, dan kapasitas. Sedangkan kamar2 dibuat menggunakan "Pendaftaran lengkap", di mana kita langsung mematok harga Rp750.000 sejak awal didaftarkan.

```
9      System.out.println(">>> UJI COBA INPUT DATA NGAWUR (Kamar 1)");
10     kamar1.setTipeKamar("Presidential");
11     kamar1.setHargaPerMalam(-10000);
12     System.out.println();
```

Dalam baris 9-12 kita menguji jika ada user yang input nya ngawur. Di sini kita tes dengan memasukkan input di tipe kamar dengan presidential, di harga per malam dengan -1000. Karena kita sudah memasang validasi di Setter sebelumnya, sistem kita pasti akan menolak data ngawur ini dan memaksanya kembali ke nilai standar (Reguler dan 50000).

```
14     System.out.println(">>> UJI COBA OVERCAPACITY (Kamar 2)");
15     kamar2.pesanKamar(4);
16     System.out.println();
17
18     System.out.println(">>> UJI COBA PESAN SESUAI ATURAN (Kamar 2)");
19     kamar2.pesanKamar(2);
20     System.out.println();
21
22     System.out.println(">>> UJI COBA DOUBLE BOOKING (Kamar 2)");
23     kamar2.pesanKamar();
24     System.out.println();
```

Di baris 14-24 ini kita menguji berbagai skenario reservasi. Di baris 15 kita mencoba paksa memesan kamar 2 dengan kapasitas 4 orang, sistem pasti menolak karena kapasitasnya hanya muat 2 orang. Di baris 19, kita memesan dengan benar, kita memesannya untuk 2 orang. Sistem akan menerima dan langsung mengunci kamarnya. Di baris 22-24, kita coba iseng memesan lagi kamar tersebut; sistem akan langsung memblokirnya untuk mencegah double booking karena kamarnya sudah terisi.

```
26     System.out.println(" UJI COBA TAGIHAN & VOUCHER");
27     double tagihanKamar1 = kamar1.hitungTotalBayar(2, "PROMO");
28     System.out.println("Total Tagihan Kamar 1 (2 Malam): Rp" + tagihanKamar1);
29
30     System.out.println();
31
32     double tagihanKamar2 = kamar2.hitungTotalBayar(4, "PROMO");
33     System.out.println("Total Tagihan Kamar 2 (4 Malam, Diskon 20%): Rp" + tagihanKamar2);
34     System.out.println();
```

Di baris 26-34 ini, kita melakukan perhitungan biaya untuk kamar 1 dan kamar 2. Untuk kamar 1, karena tagihanya hanya 2 malam, maka sistem langsung menolak kode voucher karena tidak memenuhi syarat.

Sebaliknya, pada kamar 2 karena tagihanya 4 malam, maka sistem akan langsung menghitung dan memotong total tagihannya sebanyak diskon 20%.

IV. Kesimpulan

Secara keseluruhan, proyek KamarHotel ini adalah contoh penerapan nyata yang sangat baik dari prinsip Object-Oriented Programming (OOP) untuk membangun sistem operasional yang cerdas dan tahan terhadap bug. Melalui penerapan Enkapsulasi, kita berhasil menciptakan tameng data dengan menyembunyikan atribut penting dan memasang validasi ketat pada Setter, sehingga sistem mustahil disusupi oleh kesalahan input. Selain itu, pemanfaatan Method Overloading menjadikan sistem ini sangat adaptif, ia mampu menangani berbagai skenario resepsionis di lapangan secara fleksibel, mulai dari pemesanan kilat, pencegahan overcapacity, hingga perhitungan tagihan bersyarat menggunakan voucher promo.

IV. Daftar Pustaka

Tim Asisten Laboratorium Komputer (2026). Modul Praktikum Pemrograman Berorientasi Objek – Modul 4: The Core of OOP. Program Studi Ilmu Komputer, Universitas Pertamina

Tim Asisten Laboratorium Komputer (2026). Modul Praktikum Pemrograman Berorientasi Objek – Modul 5: Encapsulation. Program Studi Ilmu Komputer, Universitas Pertamina

Dosen PBO. (2026). The Core of OOP. Universitas Pertamina

Dosen PBO. (2026). Encapsulation. Universitas Pertamina